

## PROGRAMACIÓN DE ALGORITMOS POR EQUIPO – ENTREGA 1

**MUY IMPORTANTE:** LEER DETENIDAMENTE CADA PUNTO DE ESTE DOCUMENTO YA QUE EL DESARROLLO ENTREGADO POR EL EQUIPO PERDERÁ PUNTAJE DE CALIFICACIÓN SI NO SE RESPETA CADA UNO DE LOS DETALLES ENUNCIADOS A CONTINUACIÓN.

### DESCRIPCIÓN GENERAL

Cada equipo deberá desarrollar en **Python** un programa completo de análisis y manipulación de listas, utilizando exclusivamente la plantilla `template_con_menu.py` suministrada por la cátedra. **El programa deberá respetar íntegramente la estructura de la plantilla, sin modificar su lógica de menú ni su punto de entrada.**

Esta es la **Entrega 1** de un trabajo que evolucionará en una segunda entrega. El código deberá estar organizado y documentado de forma que permita incorporar funcionalidades adicionales sin necesidad de reestructurar lo ya desarrollado.

### CONFORMACIÓN DE EQUIPOS

- Los equipos estarán conformados por **4 a 5 integrantes**.
- Cada integrante debe poder explicar y defender cualquier parte del código en instancia de evaluación.
- El nombre del archivo a entregar será: `Entrega1.py`.

### ESTRUCTURA DEL PROGRAMA

El programa operará sobre una **lista de una dimensión** de valores numéricos enteros. El menú principal contará con las siguientes opciones:

- **[1]** Generar valores
- **[2]** Mostrar valores
- **[3]** Eliminar valores repetidos
- **[4]** Filtrar valores
- **[5]** Desdoblar valores
- **[6]** Valores top N
- **[7]** Valores máximos y mínimos
- **[0]** Salir del programa

### DESCRIPCIÓN DETALLADA DE CADA OPCIÓN

**Cada opción se debe desarrollar como una función independiente.**

#### **[1] Generar valores**

- Solicitar al usuario la **cantidad de valores** a generar.
- Generar la lista utilizando el módulo `random`.
- Los valores generados deberán ser de 1 dígito, de 3 dígitos, y de 5 dígitos. Es decir, números al azar entre 0 y 9, entre 100 y 999, y entre 10000 y 99999. Las tres cantidades de dígitos tiene que tener igual probabilidad de ocurrencia, es decir que debe ser igualmente probable obtener un número de 1, 3 o 5 dígitos.
- Si ya existía una lista generada anteriormente, **informar al usuario** y solicitar confirmación antes de reemplazarla.

## [2] Mostrar valores

- Presentar los valores en formato de **tabla encolumnada y centrada**, con **10 valores por fila**. La presentación en pantalla debe respetar este formato:
  1. Título enmarcado con asteriscos (El título debe ser parametrizable ya cambiará según cuándo se llame a esta función)
  2. 10 columnas de 9 caracteres de ancho.
  3. Cada columna titulada con una letra (desde la A a la J) con un separador de 7 caracteres bajo la letra (=====)
  4. Los valores de la lista de a 10 encolumnados y centrados en sucesivas filas hasta completar la lista.
  5. El cierre del listado informando la fecha y hora de ejecución del mismo. Generar fecha y hora utilizando el módulo `time`.

Ejemplo:

```
*****
VALORES DEL JUEGO DE DATOS (TODOS LOS DATOS)
*****
  A      B      C      D      E      F      G      H      I      J
=====
 333    106    123    106     1    106     3    106     3    106
   3     106     3     106     3    106    888    106     3    106
15632   106     3    351     3    106     1    106     5    106

FIN DEL LISTADO (01-01-2026 10:15:35) *****
```

- Cada celda deberá tener un **ancho fijo** acorde a la cantidad de cifras generadas, de modo que la tabla quede siempre alineada. Si no hay valores generados, informar al usuario y volver al menú.

## [3] Eliminar valores repetidos

- Generar y mostrar una **nueva lista** a partir de la original, eliminando los valores duplicados y conservando el **orden de primera aparición** de cada valor. Implementado **a mano** con los recursos vistos en clase, sin uso de `set()`.
- La lista original **no debe modificarse**. Trabajar sobre una copia. Informar cuántos valores fueron eliminados.
- La cabecera del listado se verá así:

```
*****
VALORES DEL JUEGO DE DATOS (DATOS SIN REPETIDOS)
*****
```

## [4] Filtrar valores

- Solicitar al usuario el criterio de filtrado mediante el siguiente mensaje de entrada:

```
Ingrese el criterio de filtrado [M=Mayores que | E=Menores que | R=En rango | P=Pares |
I=Impares]:
```

- Validar que la opción ingresada sea una de las letras habilitadas (tomar como válidas tanto mayúsculas como minúsculas). Según el criterio elegido, solicitar los parámetros adicionales necesarios con sus

respectivas validaciones. Mostrar la lista resultante en el mismo formato de tabla de la opción 2. La lista original **no debe modificarse**. Los criterios disponibles son:

1. **M — Mayores que:** conserva solo los valores mayores a un umbral ingresado.
2. **E — Menores que:** conserva solo los valores menores a un umbral ingresado.
3. **R — En rango:** conserva los valores comprendidos entre un límite inferior y uno superior ingresados, incluyendo estos valores.
4. **P — Pares:** conserva solo los valores pares.
5. **I — Impares:** conserva solo los valores impares.

- La cabecera del listado se verá así:

```
*****
VALORES DEL JUEGO DE DATOS (DATOS FILTRADOS)
*****
```

## [5] Desdoblar valores

- Solicitar al usuario el criterio de desdoblamiento mediante el siguiente mensaje de entrada:

```
Ingrese el criterio de desdoblamiento [P=Pares/Impares | C=Por cantidad de cifras | U=Por
valor umbral]:
```

- Validar que la opción ingresada sea una de las letras habilitadas. Generar las listas derivadas y mostrarlas en el mismo formato de tabla de la opción 2. La lista original **no debe modificarse**. Los criterios disponibles son:

1. **P — Pares / Impares:** genera dos listas: una con los valores pares y otra con los impares.
2. **C — Por cantidad de cifras:** genera tres listas: valores de 1 cifra, de 3 cifras y de 5 cifras.
3. **U — Por valor umbral:** genera dos listas: valores hasta un valor umbral, y valores por encima de él.

- La cabecera de cada listado se verá así:

```
*****
VALORES DEL JUEGO DE DATOS (DATOS DESDOBLADOS)
*****
```

## [6] Valores top N

- Solicitar al usuario la cantidad **N** de valores a mostrar en el ranking (por ejemplo, top 10). Identificar los **N valores más altos únicos**: si un valor aparece varias veces en la lista, debe figurar **una sola vez** en el ranking. Presentar el ranking numerado de mayor a menor, con título enmarcado pero con los valores en una sola columna.
- Si la cantidad de valores únicos es menor a N, mostrar todos los disponibles e informar la situación. Implementado **a mano** con los recursos vistos en clase, sin uso de `sorted()`, `set()` ni `max()`.
- La cabecera del listado se verá así:

```
*****
VALORES DEL JUEGO DE DATOS (DATOS TOP N)
*****
```

## [7] Valores máximos y mínimos

- Identificar el valor **máximo** y el valor **mínimo** de la lista. Implementado **a mano** con los recursos vistos en clase, sin uso de `max()` ni `min()`.
- Mostrar la lista completa (mismo formato que la opción 2) con máximos y mínimos **destacados simultáneamente** con símbolos distintos: máximo entre corchetes `[valor]`, mínimo entre paréntesis `(valor)`.

Ejemplo:

```
*****
VALORES DEL JUEGO DE DATOS (DATOS MÁXIMOS Y MÍNIMOS [máx] (mín) <máx/mín>)
*****
  A      B      C      D      E      F      G      H      I      J
=====
  333    106    123    106    ( 1 )    106     3     106     3     106
   3     106     3     106     3     106    888     106     3     106
[15632]  106     3     351     3     106    ( 1 )    106     5     106
```

- Si hay múltiples ocurrencias del máximo o del mínimo, **todas deben destacarse**. Si un valor es a la vez máximo y mínimo (todos los elementos son iguales), utilizar la notación `<valor>`
- La cabecera del listado se verá como la del ejemplo anterior.

## REQUISITOS TÉCNICOS PARA EL DESARROLLO

- El código se desarrollará en un archivo con nombre `Entrega1.py`, utilizando exclusivamente la plantilla `template_con_menu.py` suministrada. No modificar su estructura base, sólo se debe codificar donde aparecen los 3 puntos ...
- La **cantidad mínima de valores** a generar será de 2 números.
- Solo se permite la importación de los módulos `random` y `time`, ambos vistos en clase.
- Cada opción del menú debe implementarse en una **función separada**. Las funciones auxiliares reutilizables (presentación en tabla, validación de entradas, etc.) deben separarse en sus propias funciones. Todas las declaraciones `def` deben aparecer contra el margen izquierdo.
- Todo input de datos debe contar con **validación** de rango y/o tipo. La validación de enteros debe realizarse mediante `.isdigit()` sobre el string ingresado, sin usar `try/except`. Las validaciones de letras de criterio deben aceptar mayúsculas y minúsculas.
- Completar el **encabezado del archivo** con título, fecha, autores y descripción. Cada función debe incluir un **docstring** con su descripción, parámetros y valor de retorno. Incluir comentarios explicativos en los bloques de lógica no trivial.

## INCLUSIONES Y EXCLUSIONES

Dado que este desarrollo evolucionará en una próxima entrega, en esta oportunidad se deben aplicar sólo las características solicitadas.

- El código **SÍ** debe incluir las siguientes características:
  1. Uso de los módulos `random` y `time`.
  2. Validación en todos los inputs de ingreso de datos.
  3. Documentación básica del código mediante comentarios.

4. Documentación de todas las funciones mediante docstring.
  5. Una función por opción de menú, más las funciones auxiliares necesarias.
- El código **NO** debe incluir las siguientes características:
    1. Importación de módulos distintos a `random` y `time`.
    2. Gestión de excepciones (`try / except`).
    3. Uso de funciones built-in de ordenamiento: `sorted()`, `list.sort()`.
    4. Uso de `set()` para eliminación de duplicados o cualquier otra operación sobre listas.
    5. Uso de `max()`, `min()`, `sum()`, `count()` u otras funciones built-in que resuelvan directamente la lógica pedida.
    6. Slicing de listas (`lista[a:b]`) para invertir, tomar partes de una lista, realizar cálculos con sus elementos, o seleccionar elementos que cumplan una condición (NOTA: Sí está permitido usar la forma `lista[:]` para clonar una lista)
    7. Comprensiones de listas o listas por comprensión (`[x for x in lista]`).
    8. Uso de recursividad.
    9. Uso de expresiones regulares.
    10. Gestión de archivos.
    11. Declaración de funciones dentro de otras funciones (sólo deben aparecer declaraciones `def` contra el margen izquierdo).
    12. Uso de instrucciones: `continue` / `assert` / `class` / `global` / `lambda` / `nonLocal` / `yield` / `async` / `eval` / `isinstance`.

## CRITERIOS DE EVALUACIÓN

- **Funcionamiento correcto** de cada opción del menú.
- **Calidad y claridad del código:** nombres de variables descriptivos, indentación correcta, comentarios.
- **Validaciones:** robustez ante entradas incorrectas del usuario.
- **Modularización:** uso adecuado de funciones, sin repetición de código.
- **Presentación en pantalla:** tabla correctamente formateada, alineada y centrada.
- **Documentación:** encabezado completo, docstrings y comentarios en el código.
- **Cumplimiento de exclusiones:** el uso de técnicas o funciones no permitidas resultará en descuento de puntaje.

## FORMA DE ENTREGA

- Subir el archivo `Entrega1.py` a la tarea correspondiente en Teams antes de la fecha de vencimiento.
- El archivo debe ejecutarse sin errores desde Thonny u otro IDE estándar.